

Assignment M1.1 - Getting to know JavaScript (Graded)

Student Name: OLABISI AFOLABI

1. Why is JavaScript considered a loosely typed programming language?

JavaScript is considered loosely typed because it doesn't force you to declare what type of data you're storing in a variable. You can put a number in a variable, then later change it to text, and JavaScript won't complain. This makes it more flexible (easier to start with) but can sometimes cause confusion because JavaScript might automatically change data types for you without asking. This flexibility makes JavaScript easier to write for beginners but can lead to unexpected type related bugs if not handled carefully.

Example:

Javascript

```
Let myVariable = 10           // this is a number
myVariable = "hello";         // this is a text. Javascript allow both.
```

2. What is the difference between 'var' and 'let' when declaring variables in JavaScript?

var and let are two different rules for where variables can be seen.

- a. Var = "function room" visibility – if you declare it in a function, it can be seen anywhere in that function
- b. Let = "block room" visibility – if you declare it inside curly braces {}, it can only be seen inside those braces.

In summary, var is function-scoped, meaning that, it is accessible anywhere within the function it is declared and let is block-scoped, meaning that, it is only accessible within the nearest enclosing {} block (like inside an if statement or loop).

In general, and usage, **let** is safer because it keeps your variables contained. With **var**, you might accidentally change a variable from somewhere you didn't expect to.

3. What does ECMAScript refer to? Is JavaScript the same as ECMAScript?

ECMAScript is the official standard specification for scripting languages, originally created to standardize JavaScript. JavaScript is an implementation of the ECMAScript standard, with additional features provided by the host environment (like web browsers).

So, JavaScript is based on ECMAScript but is not the same— ECMAScript seems like the official recipe book for JavaScript. JavaScript is the **actual meal** you cook using that recipe book. Different chefs (like Chrome, Firefox, or Node.js) might add their own little touches, but they all follow the same basic recipe from ECMAScript.

In summary:

ECMAScript = The official rules of basketball

JavaScript = The actual basketball game being played

4. What are reserved words, and why can't we use them to name variables and functions?

Reserved words are JavaScript's special toolbox words. Words like if, for, function, return, etc. They are part of the language syntax and cannot be used as identifiers (variable or function names) because doing so would create uncertainty for the JavaScript engine when parsing code. For example, naming a variable if would confuse the engine when it encounters an if statement.

5. What is a conditional statement, and how does an if-else block work?

A conditional statement allows different blocks of code to execute based on whether a condition evaluates to true or false. It's like a **decision point** in your code: "If this is true, do that. Otherwise, do something else."

An if-else block works like this:

The condition inside the if is evaluated.

If it is true, the code inside the if block runs.

If it is false, the code inside the else block runs (if an else is provided).

Example:

javascript

```
if (age >= 18) {  
    console.log("Adult");  
} else {  
    console.log("Minor");  
}
```

6. Explain the difference between == and === in JavaScript. Why is one generally preferred over the other?

== (loose equality) compares values with type coercion, meaning JavaScript will try to convert types to make them match before comparing.

=== (strict equality) compares values without type coercion; both value and type must be the same.

Example:

5 == "5" → true (type coercion converts string "5" to number 5) Are these kind of the same? JavaScript tries to make them match.

5 === "5" → false (different types) Are these exactly the same? (No changes allowed)

=== is generally preferred because it avoids unexpected results caused by automatic type conversion, making code more predictable and less error prone.

7. What will the following code output, and why?

javascript

```
if (0) {  
    console.log('Hello');  
} else {  
    console.log('World');  
}
```

Output:

text

World

Why:

JavaScript treats some values as "false-like" (falsy) in if-statements. The number **0** is one of these falsy values. So when JavaScript checks if (0), it's like asking "Is this true?" and 0 means "no."